

que 8
04

Instrucción: A continuación se presentan 4 fragmentos de código que resuelven problemas distintos. Para cada uno, debes:

1. **Identificar** la técnica utilizada en el código proporcionado.
2. **Justificar** por qué esa técnica es la correcta para ese problema específico.

```
struct Actividad {  
    int inicio;  
    int fin;  
};  
  
// Función auxiliar para el criterio de ordenamiento  
bool compararPorFin(Actividad a, Actividad b) {  
    return a.fin < b.fin;  
}  
  
int seleccion_tareas(vector<Actividad>& lista_actividades) {  
    // 1. Criterio de orden: El primero en terminar es el que menos estorba  
    sort(lista_actividades.begin(), lista_actividades.end(), compararPorFin);  
  
    // 2. Tomamos la primera actividad porque es la que termina más rápido  
    Actividad ultima_seleccionada = lista_actividades[0];  
    int conteo = 1;  
  
    // 3. Revisamos el resto una por una  
    for (size_t i = 1; i < lista_actividades.size(); i++) {  
        Actividad actividad_actual = lista_actividades[i];  
  
        // DECISIÓN: ¿Esta actividad empieza después de que terminó la anterior?  
        if (actividad_actual.inicio >= ultima_seleccionada.fin) {  
            // Si cabe en el tiempo libre, la tomo de inmediato  
            ultima_seleccionada = actividad_actual;  
            conteo = conteo + 1;  
        }  
    }  
    return conteo;  
}
```

```

struct Objeto {
    double valor;
    double peso;
    double precio_por_kilo;
};

bool compararPorRatio(Objeto a, Objeto b) {
    return a.precio_por_kilo > b.precio_por_kilo;
}

double carga_objetos(vector<Objeto>& saco, double capacidad_libre) {
    // 1. Calculamos la "densidad de valor" (¿cuánto oro vale cada kilo?)
    for (auto &objeto : saco) {
        objeto.precio_por_kilo = objeto.valor / objeto.peso;
    }

    // 2. Priorizamos lo más caro por gramo de mayor a menor
    sort(saco.begin(), saco.end(), compararPorRatio);
    double valor_total = 0;

    // 3. Llenamos el saco con lo más valioso primero
    for (const auto &objeto : saco) {
        if (objeto.peso <= capacidad_libre) {
            // Cabe completo: lo tomo todo
            capacidad_libre -= objeto.peso;
            valor_total += objeto.valor;
        } else {
            // No cabe completo: tomo solo el trozo que quepa
            valor_total += (objeto.precio_por_kilo * capacidad_libre);
            break; // TERMINAR BUCLE (el saco está lleno)
        }
    }
    return valor_total;
}

```

```

struct Carretera {
    int destino;
    int precio;
};

int conectar_red(int total_ciudades, vector<vector<Carretera>>& adj, int ciudad_inicial) {
    set<int> ciudades_conectadas;
    ciudades_conectadas.insert(ciudad_inicial);
    int costo_total = 0;

    // MIENTRAS existan ciudades sin conectar
    while (ciudades_conectadas.size() < total_ciudades) {
        // 1. Miramos todas las carreteras que salen de nuestras ciudades conectadas
        // hacia las que todavía están aisladas.
        Carretera mas_barata = buscar_arista_minima(ciudades_conectadas, adj);

        // 2. DECISIÓN: Tomamos la conexión más barata disponible AHORA
        costo_total += mas_barata.precio;

        // 3. Marcamos la ciudad del otro extremo como conectada
        ciudades_conectadas.insert(mas_barata.destino);
    }
    return costo_total;
}

```

Algoritmo (El algoritmo de búsqueda por...

```
int entregar_cambio(int monto_objetivo, vector<int>& billetes_disponibles) {  
    // 1. Ordenamos el dinero de mayor a menor valor  
    sort(billetes_disponibles.rbegin(), billetes_disponibles.rend());  
    int billetes_entregados = 0;  
  
    // 2. Intentamos usar siempre el billete más grande primero  
    for (int valor_billete : billetes_disponibles) {  
        // ¿Cuántas veces cabe este billete en lo que falta por pagar?  
        int cantidad = monto_objetivo / valor_billete;  
  
        if (cantidad > 0) {  
            // DECISION: Tomamos tantos billetes de este valor como podamos  
            billetes_entregados += cantidad;  
            monto_objetivo -= (cantidad * valor_billete);  
        }  
    }  
    return billetes_entregados;  
}
```